Software: Practice and Experience

WILEY

RESEARCH ARTICLE OPEN ACCESS

Orchestrating TinyML Soft Sensors With Data Streams: A Hybrid Cloud-Edge Architecture for IoT Scenarios

Antonio Jesús Chaves[1] | Cristian Martín[1] | Adnan Shahid[2] | Manuel Díaz[1]

[1]ITIS Software, University of Málaga, Málaga, Spain | [2]IDLab, Department of Information Technology, Ghent University – Imec, Ghent, Belgium

Correspondence: Antonio Jesús Chaves (chaves@uma.es)

Received: 2 March 2026 | **Revised:** 1 September 2026 | **Accepted:** 1 October 2026

Keywords: edge computing | Kafka-ML | model orchestration | soft sensors | TinyML

ABSTRACT

Background: The deployment of soft sensors, data-based models that estimate unobservable or costly variables from available sensor data, remains limited in IoT infrastructures by the resource constraints of IoT devices and the complexity of managing model lifecycles at scale.

Objective: This work presents a novel architecture that enables the end-to-end transformation, distribution and execution of soft sensors on IoT embedded devices.

Methods: The proposed approach leverages Kafka-ML, a framework for managing ML models and data streams, which has been extended to orchestrate the end-to-end lifecycle of soft sensors in real time via IoT data streams. By enabling local inference on IoT devices, the system reduces reliance on cloud connectivity during operation: while initial deployment, model updates, and lifecycle orchestration via Kafka-ML require network access, local inference allows the estimation of complex variables even when subsequent connectivity is unreliable or intermittent. The approach has been evaluated in two representative scenarios, non-intrusive load monitoring (NILM) and water quality monitoring, each with different demands regarding communications overhead and accuracy.

Results: The automated quantization pipeline yields negligible accuracy degradation for the NILM scenario (MAE shifted from 2.836 to 2.837) and an acceptable performance trade-off for water quality monitoring (MAE rose from 0.53 to 0.82 mg/L), while reducing inference latency by up to 4.6× compared to remote inference.

Conclusions: These results demonstrate the framework's capability to automate the deployment of distributed, autonomous, and privacy-preserving soft sensors across heterogeneous monitoring scenarios.

1 | Introduction

Smart environments are becoming increasingly rich ecosystems composed of interconnected, heterogeneous devices. From water level sensors and smart thermostats to lights and energy meters, these devices continuously generate a vast amount of data [1]. This data can be highly valuable for automating behaviors, detecting anomalies, and improving efficiency. However, a significant challenge arises when critical variables cannot be measured directly due to high sensor costs, physical constraints, or harsh environments. This has led to the adoption of soft sensors: predictive models that act as 'virtual sensors' by estimating complex variables from easier-to-measure physical data [2]. Consequently, these soft sensors provide insights to optimize real-time control and enhance decision-making [3].

Traditionally, machine learning (ML) applications for soft sensing have relied on transferring data to the cloud, where models

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). *Software: Practice and Experience* published by John Wiley & Sons Ltd.

are trained and inference is performed. However, while training might require the computational resources of the cloud, performing inference in the cloud can introduce latency, increases dependence on stable connectivity, and may raise privacy concerns, especially when dealing with sensitive data [4]. Recent developments in Artificial Intelligence (AI) have enabled a shift in this paradigm. Lightweight inference frameworks such as LiteRT for Microcontrollers (previously Tensorflow Lite for Microcontrollers, TFLM) [5, 6] now allow smart devices to run the models and carry out the inference locally without relying on external computation, a paradigm known as Tiny Machine Learning (TinyML). It is important to distinguish between management and operation phases in this paradigm. While the initial data ingestion for training, as well as the delivery of OTA updates and model binaries, strictly require network connectivity to the cloud, the execution phase does not. Once deployed, the TinyML model operates autonomously, enabling continuous local inference regardless of external network stability. Nonetheless, deploying and updating these models across dynamic or distributed IoT scenarios remains a logistical and technical challenge. Ensuring that the right model version reaches the correct device, at the right time, in an efficient and reliable way, represents a significant operational challenge.

Despite these advances, the process of deploying, updating, and managing TinyML models across numerous IoT devices still relies heavily on manual configuration and flashing procedures. This manual intervention leads to difficulties in scalability and reproducibility, especially when dealing with heterogeneous hardware or dynamic environments where models must adapt to evolving data streams. Therefore, an automated pipeline that unifies model transformation, quantization, and deployment is essential to ensure that model updates can be delivered seamlessly and consistently to IoT devices without human intervention.

On the other hand, one of the main challenges in the domain of TinyML for sensing is the misalignment between real-time data streaming and the management of model binaries on-device. Traditionally, these processes have been addressed independently, making it difficult to efficiently implement local inference solutions and dynamically update models at embedded devices at scale.

A modern framework that enables the native distribution, orchestration, and execution of ML models in distributed data-driven architectures is Kafka-ML [7]. Kafka-ML is an open-source tool designed to orchestrate the full lifecycle of ML/AI workflows using data streams. Through its web interface, Kafka-ML enables users to define, train, evaluate, and deploy machine learning models using real-time data streams.

Recently, Kafka-ML has been validated for soft sensors compatibility [8], significantly extending its applicability to resource-constrained devices commonly used in IoT deployments. This compatibility allows Kafka-ML to act as a central controller in stream-based systems, while still interfacing with edge devices that communicate via MQTT. So far these soft sensor predictions have been carried out in the cloud. In this work, we propose a novel architecture and automated pipeline integrated into Kafka-ML that enables the end-to-end management and deployment of TinyML models. While the architecture functions as a general-purpose IoT TinyML orchestration framework capable of handling diverse edge AI tasks, this paper focuses on its application to soft sensors. To achieve this, Kafka-ML has been updated to interact directly with IoT devices through a new module that manages the model transformation pipeline and Over-The-Air (OTA) updates.

The main contributions of this work are summarized as follows:

- **Unified architecture for embedded soft sensors:** A hybrid cloud-edge TinyML framework that orchestrates the end-to-end lifecycle of virtual sensors, combining cloud-based model management with autonomous on-device inference.
- **Automated model transformation pipeline:** A conversion pipeline is implemented to transform TensorFlow models into TFLM format, including 8-bit integer quantization based on training data at Apache Kafka, optimizing models for efficient IoT inference.
- **Integrated OTA model deployment:** The framework supports secure, wireless deployment and updates of ML models to IoT devices without manual flashing and ensuring timely model delivery.
- **Extension of Kafka-ML for direct TinyML interaction:** Kafka-ML is extended to directly interface with IoT-enabled devices via MQTT, enabling centralized orchestration of model training, deployment, and update operations across distributed IoT environments.
- **Demonstration in two smart environment scenarios:** The proposed approach is validated through trace-driven experiments using physical edge hardware and empirical datasets, demonstrating automated, low-latency orchestration of soft sensor workflows for IoT inference and dynamic model updates.

The rest of the paper is structured as follows. Section 2 presents the related work and their differences against our approach. Section 3 presents the system architecture and implementation details. Section 4 shows the components and the pipeline of a soft sensor model deployment using Kafka-ML. Section 5 provides an evaluation over the different test cases. Finally, Section 6 provides a concise overview of the study and explores potential avenues for future growth.

2 | Background and Related Work

2.1 | TinyML Frameworks and Optimization Techniques

Recent advances in TinyML are enabling the deployment of intelligent models on low-power and resource-constrained devices. These technologies allow for on-device inference without cloud dependency, improving privacy, reducing latency, and enabling continuous, localized operation in environments such as energy monitoring, environmental sensing, security, and health. This section explores key contributions in the field, highlighting recent frameworks, optimization techniques, and

practical applications that demonstrate the growing role of TinyML in enabling intelligent and autonomous behavior in different scenarios.

Schizas et al. [9] provide a comprehensive systematic review of TinyML approaches, outlining its evolution as a response to the increasing demand for on-device intelligence in large-scale IoT systems. The authors discuss core benefits such as reduced latency, energy efficiency, and enhanced privacy, and present a taxonomy of hardware platforms and software tools including TFLM, Edge Impulse, and STM32Cube.AI. Application areas covered in the review, such as smart agriculture, environmental monitoring, and home automation, underline the relevance of TinyML for decentralized smart systems. Furthermore, the evaluation of such edge ecosystems is increasingly guided by standardized metrics, such as the MLPerf Tiny benchmark [10], which establishes a methodological canon for measuring latency, energy, and accuracy trade-offs across constrained devices.

DTMM [11] is a full-stack software library based on a novel filterlet-based pruning technique and a compact weight storage format (FWCS) designed to improve the efficiency of model deployment on highly constrained microcontrollers. The library includes a SIMD-compatible convolution operator and an instruction-level acceleration mechanism, enabling real-time inference with reduced memory footprint and execution time.

MinUn [12] is a TinyML framework designed for ultra-low-memory microcontrollers, addressing core challenges such as memory fragmentation and flexible number representation. MinUn combines a bitwidth assignment algorithm and a memory layout strategy to achieve high compression rates while preserving accuracy. Its general-purpose design supports efficient edge inference, opening the door for deployment in environments that demand compact, high-performance models.

MicroFlow [13] is a Rust-based inference engine targeting highly constrained embedded systems. The framework introduces a statically allocated memory model, compiler-time optimizations, and a paging mechanism for executing large neural layers on devices with as little as 2 kB of RAM. Its modular, memory-safe design supports real-time tasks such as voice and presence detection, making it a compelling option for secure and reliable edge AI applications.

Sudharsan et al. [14] tackle the challenge of model updates by proposing OTA-TinyML, a system for OTA deployment of machine learning models on low-power IoT devices. Using secure HTTPS protocols and dynamic memory loading, the framework enables remote updates of models stored in various memory units (e.g., Flash, EEPROM, SD). Its hardware-agnostic design supports flexible model management across a range of smart home devices without requiring physical access.

The AI-on-the-edge-device project [15] offers a practical, open-source solution for retrofitting analogue utility meters using the ESP32-CAM. The system employs local image acquisition and digit recognition using TFLM, supporting OTA updates, MQTT integration, and compatibility with platforms like Home Assistant. Its low cost, modularity, and independence from cloud services illustrate the potential of TinyML for autonomous sensing in residential environments.

Beyond academic frameworks, production-grade, proprietary enterprise ecosystems such as Edge Impulse [16] and Nordic Edge AI Lab [17] have heavily consolidated the deployment of TinyML applications in industrial contexts. These platforms offer sophisticated pipelines for dataset curation, model profiling, and optimized code generation tailored for embedded targets.

Although existing TinyML frameworks provide essential contributions such as memory-efficient inference, secure OTA updates, and hardware-specific model optimizations, there exist some gaps in lifecycle and deployment orchestration, rather than in the on-device inference mechanism itself. Crucially, commercial ecosystems are closed-source, restricting self-hosted infrastructure deployments and preventing deep framework customization. Furthermore, they lack native integration with real-time pub/sub stream orchestrators (e.g., Apache Kafka) to manage the continuous end-to-end lifecycle. While ongoing TinyML works successfully demonstrate the broader lifecycle of off-device training and on-device inference (e.g., wearable activity recognition pipelines [18]), both academic and industrial platforms predominantly rely on monolithic firmware architectures for model delivery and updates.

In these approaches, model recalibration typically relies on manual data handling or offline pipeline execution using static datasets rather than dynamic data streams like our approach. The resulting models are then compiled directly into static C/C++ structures within a monolithic firmware binary target. This necessitates a full device flash or localized, point-to-point Device Firmware Updates (DFU) for any update, completely lacking the capability to execute decoupled, hot-swappable OTA updates of the machine learning model or its execution logic independently. This dependence on manual intervention or OS-halting DFUs scales linearly ($O(N)$) with the fleet size, complicating large-scale maintenance, introducing human error, and preventing rapid adaptation when models or data evolve.

This is particularly critical for soft sensors, where the relationship between input signals and the estimated variable may drift over time, requiring frequent model recalibration and seamless redeployment to maintain sensor accuracy. As summarized in Table 1, most frameworks primarily focus on optimizing inference for resource-constrained devices but offer limited support for automated deployment, real-time data streams, or completely independent logic updates.

By contrast, our open-source approach extends this conventional pipeline with an automated, event-driven workflow that dynamically integrates live data streams for quantization calibration and decouples the ML model from the device firmware. By embedding the model transformation plane directly within an active Kafka streaming architecture and utilizing a scripting engine over a stable host OS (Tasmota), the proposed architecture separates the TFLM model binary and the runtime execution script (Berry script) from the underlying host OS. This separation in the deployment mechanism enables automated, constant-time ($O(1)$) hot-swappable OTA updates of the model and execution logic simultaneously, achieving seamless

TABLE 1 | Comparison of TinyML frameworks and related approaches.

Approach	IoT platform/firmware	OTA	Data streaming	Open source	Frameworks supported
Schizas et al. [9]	Various MCUs/C, Python	×	×	✓	Multiple Frameworks
DTMM [11]	Cortex-M55 and Helium SIMD/C	×	×	×	Custom
MinUn [12]	Ultra-low-memory MCUs/C++	×	×	✓	TensorFlow, PyTorch, ONNX
MicroFlow [13]	Various MCUs/Rust	×	×	✓	MicroFlow vs. TFLM
OTA-TinyML [14]	Various MCUs/Arduino	✓	×	✓	TFLM
AI-on-the-edge [15]	ESP32-CAM/C++	✓	MQTT	✓	TFLM
Edge Impulse [16]	Various MCUs/C++	3rd Party	REST APIs	×	Multiple Frameworks
Nordic Edge AI [17]	nRF MCUs/C	✓ (DFU)	×	×	Neuton and Axon (TFLM)
Our Approach	**ESP32-based devices/Tasmota (C)**	**✓ (Decoupled)**	**Kafka + MQTT**	✓	**TFLM**

fleet orchestration without firmware recompilation or system downtime.

2.2 | TinyML Soft Sensors Applications

Several studies have explored the implementation of soft sensors for health and environmental monitoring. Ooko et al. [19] explore a health-focused smart home use case with an edge AI framework for respiratory disease detection from exhaled breath audio signals. The system runs on embedded devices and leverages lightweight models to enable decentralized, continuous health monitoring with low power requirements, demonstrating the feasibility of soft sensors for biomedical applications within smart homes.

Tsoukas et al. [20] propose a TinyML-based gas leakage detection system designed for real-time, low-power indoor air quality monitoring in smart homes. The system utilizes Arduino Nano microcontrollers combined with gas sensors to detect harmful concentrations of gases such as ammonia and smoke. By performing on-device inference with gas sensors, their solution enables low-cost, energy-efficient, and private indoor air quality monitoring for smart homes, without relying on cloud connectivity.

Atanane et al. [21] propose a TinyML-based solution for real-time water leakage detection in smart buildings. The system uses acoustic vibration data collected from pipelines and applies deep learning techniques to detect leak conditions. Model quantization combined with scalogram features allows for efficient, on-device leak detection in support of water management, underscoring the suitability of TinyML for robust, cloud-independent smart building scenarios.

In the domain of energy management, soft sensing plays a vital role in disaggregating consumption data. Németh and Tornai [22] present a smart plug prototype designed to perform fast and accurate on-device electrical load classification using TinyML. The system combines two microcontrollers and introduces a dimmer-based method for manipulating the AC signal to extract characteristic load responses. On-device inference with TFLM demonstrates that advanced models can run efficiently on memory-constrained microcontrollers. As a result, the system achieves low latency, reduced communication overhead, and high reliability, supporting practical smart grid and energy management applications.

While all the use cases achieve efficient on-device model deployment, they impose a considerable dependency on user intervention for calibration, feature selection, or customization. This reliance on technical expertise can impede widespread adoption among typical smart home users and limits the ease of integrating such solutions in diverse, real-world environments.

Pattanayak et al. [23] and Pattnaik et al. [24] present different edge-intelligent soft sensors for real-time estimation of Chemical Oxygen Demand and Biological Oxygen Demand respectively. The system uses easily measured water parameters as inputs and evaluates several machine learning models. Training is performed in the cloud, while low-latency inference runs on a Raspberry Pi (RPi) 3B+, reducing system costs by removing the need for expensive sensors and delivering on-device predictions faster than cloud-based inference. This approach demonstrates the suitability of lightweight ML models and edge deployment for decentralized, real-time environmental monitoring scenarios.

Pereira et al. [25] propose an energy-efficient TinyML model for real-time water potability classification on constrained embedded devices. Their approach leverages only those water quality features accessible via electronic sensors such as pH or conductivity, among others, and implements a classifier optimized for deployment on the ESP32 microcontroller. Compared to cloud-based alternatives, their embedded solution achieves significant reductions in memory usage, inference time, and energy consumption, enabling multi-year autonomous operation on battery-powered nodes.

TABLE 2 | Comparison of TinyML-based soft sensor mappings and deployment workflows in smart environments.

Approach	Physical input (sensors)	Target virtual variable	Hardware/MCU	Deployment workflow
Ooko et al. [19]	VOC and Audio sensors	Respiratory disease status	STM32F401CE	Manual Flash
Tsoukas et al. [20]	MQ Gas sensors	Harmful gas concentration	Arduino Nano	Manual Flash
Atanane et al. [21]	Acoustic vibrations	Water leakage detection	Arduino Nano	Manual Flash
Németh & Tornai [22]	Modified AC signal	Electrical load classification	RPi Pico and ESP32	Manual Flash
Pattanayak et al. [23]	Standard water params (e.g., pH)	Chemical Oxygen Demand (COD)	RPi 3B+	Manual Flash
Pattnaik et al. [24]	Standard water params (e.g., pH)	Biological Oxygen Demand (BOD)	RPi 3B+	Manual Flash
Pereira et al. [25]	Standard water params (e.g., pH)	Water potability	ESP32	Manual Flash
Cerioli et al. [26]	Optical/Spectral data	Microplastic presence	Multiple MCUs	Manual Flash
Our Approach	**Agnostic (e.g., Active Power, pH)**	**Agnostic (e.g., Disaggregated Load, NO_3^-)**	**ESP32**	**Automated OTA via Kafka-ML**

Cerioli et al. [26] propose a dedicated framework for microplastic detection on edge devices with tight resource constraints. They introduce NeuralCasting, a Python-based compiler that translates quantized ONNX models directly into native C code, removing the need for inference engines like TFLM or ONNX Runtime. This approach reduces memory use, latency, and energy consumption, while increasing portability across hardware. Tested on several microcontrollers, NeuralCasting delivers faster and more efficient inference with minimal resource usage, showing its value for real-time, low-power environmental monitoring.

Existing water quality monitoring solutions deliver efficient soft sensor inference on affordable hardware, yet they often overlook crucial aspects such as OTA model updates, device status monitoring, and system lifecycle management. The absence of mechanisms for remote model update or automated health diagnostics can hinder long-term reliability and limit scalability, posing significant barriers for widespread, maintenance-free deployment in real-world conditions.

Table 2 details the physical-to-virtual variable mappings across recent TinyML soft sensor applications. While existing solutions successfully demonstrate efficient on-device inference for specific physical targets, their operational lifecycles rely almost exclusively on manual firmware flashing. In contrast, our proposed solution is domain-agnostic and provides an end-to-end TinyML management architecture. By decoupling the ML model from the device firmware, the system enables real-time data ingestion, dynamic model retraining [27], and automated OTA deployment, providing an adaptive solution for managing TinyML models in heterogeneous smart environments.

To sum up, although TinyML has gained traction across domains such as air quality monitoring, energy management, and environmental monitoring, a significant gap persists in the availability of practical methodologies for integrating ML inference directly with data streams, ensuring functional operation of soft sensors at the edge. Specifically, limited research addresses the challenges of deploying and maintaining models that operate on real-time data streams at embedded devices. The following sections highlight the system-level design considerations, deployment challenges, and solutions developed to enable efficient and reliable on-device intelligence.

3 | System Architecture for Stream-Driven TINYML Orchestration

The proposed architecture is based upon our open-source framework Kafka-ML.[‡] Kafka-ML orchestrates machine learning pipelines in containerized environments built on Kubernetes. It leverages Apache Kafka as the core message bus for real-time data processing, enabling the coordination of ML pipelines with data streams across multiple microservices, allowing the inclusion of new components seamlessly.

Although originally conceived for cloud-based ML workflows, Kafka-ML has been extended in this work to enable seamless integration of ML inference tasks into resource-constrained devices. This evolution aligns with the growing demand for real-time, local decision-making in smart environments, where latency, bandwidth, energy efficiency and privacy are critical factors. In this context, Kafka-ML provides a centralized environment for training ML models. Once trained, these models can be converted into lightweight formats, and then deployed to devices. This transformation effectively turns a standard IoT node into a specialized soft sensor node, enabling local inference without constant cloud connectivity.

To support this integration, the Kafka-ML architecture (Figure 1) has been extended with a new module, the IoT Manager. This module is designed to register and manage IoT devices at the backend, transform models into TFLM format as well as to quantize the models to 8-bit integer format, all in an automated pipeline.

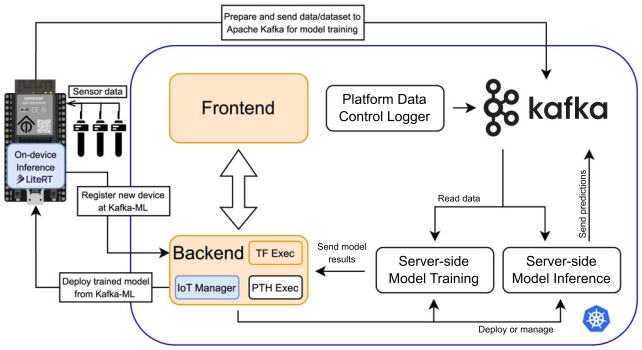


FIGURE 1 | Kafka-ML microservices architecture. **Orange component**: Modified; **Blue component**: Added.

To clearly delineate the scope of the architecture, the system comprises both reused foundations and novel developments. From prior work, we reuse the core Kafka-ML infrastructure for cloud-based training and data routing, alongside third-party open-source tools, specifically the Tasmota firmware and the TFLM engine. The novel contributions developed for this architecture include: (1) the Kafka-ML extensions (*tfexecutor*, *backend*, and *frontend* adaptations) to support automated edge orchestration and quantization, and (2) the device-side execution logic that manages OTA model updates, synchronizes sensor streams with the inference engine, and handles MQTT-based actuation without human intervention.

This integration enables Kafka-ML to function not only as a cloud model orchestrator but also as a central platform for managing ML inference workloads in embedded devices. It supports the ML lifecycle from cloud-based model training and orchestration to low-power, local inference on IoT devices. This completes the end-to-end ML deployment lifecycle, providing a fully open-source, modular, and automated solution for real-time automation and monitoring in smart environments.

The proposed architecture facilitates an adapted pipeline for soft-sensing applications, depicted in Figure 2. First of all, IoT devices should be declared and configured within the Kafka-ML framework. Then, data from IoT devices (e.g., water level, temperature, or power meters) is streamed into the desired Kafka topics (Step 1). Then, Kafka-ML trains the defined ML models (previously defined at Kafka-ML Web App) using those data streams (Step 2). Later on, the trained and chosen model is converted, exported, and integrated into the embedded runtime of the target microcontroller (Step 3). Finally, local inference is executed within the device, enabling it to act as a standalone soft sensor that autonomously performs tasks such as anomaly detection or pattern recognition (Step 4).

By embedding intelligence at the IoT layer, this architecture reduces latency and increases privacy and autonomy by enabling devices to perform actions, such as closing a gate or activating a motor, based on predictive results. This approach transforms simple IoT devices into intelligent agents driven by real-time soft-sensing data. Crucially, this architecture effectively uncouples the management lifecycle from the operational lifecycle. Kafka-ML acts as the centralized control plane, requiring an active MQTT connection for structural updates, telemetry collection, and OTA model distribution. However, the data plane, which encompasses the actual soft sensing and actuation (Step 4), is fully decentralized. If the network link to the Kafka-ML broker is severed, the device retains its current model and logic, ensuring uninterrupted local control and inference until connectivity is restored. This approach therefore serves as a powerful bridge between cloud-native ML development and practical TinyML deployment in real-world, low-power environments.

4 | Automated Model Transformation and Seamless IOT Deployment Pipeline

The integration of TinyML models into resource-constrained environments requires a structured orchestration of hardware configuration, data acquisition, and model optimization. In this work, IoT devices utilize Tasmota [28] as the primary firmware to facilitate seamless OTA updates of both logic and models. While the general architecture and the overall pipeline (illustrated in Figure 2) were introduced in the previous section, the following subsections provide a detailed description of the four key stages depicted.

4.1 | Soft-Sensor Design and Registration

To model the soft-sensing framework within our architecture, let the physical sensing space be defined as a set of observable variables $X = \{x_1, x_2, \ldots, x_n\}$ captured directly by the embedded hardware (e.g., global active power, intensity). The objective of the soft sensor is to continuously estimate a virtual, unobservable, or costly target variable $y_v \in \mathbb{R}$ (e.g., specific appliance load, NO_3^- concentration) without relying on a dedicated physical sensor for y_v.

This mapping is achieved through a TinyML model parameterized by θ, such that $y_v \approx F_\theta(X)$. Unlike traditional cloud-based inference, our architecture deploys F_θ directly to the edge. This formulation guarantees that once the model weights θ are optimized and quantized via the Kafka-ML pipeline, the continuous mapping $X \to y_v$ occurs entirely within the local microcontroller's execution loop, fully decoupling the virtual sensor's operation from network availability.

To instantiate this physical-to-virtual mapping, the process begins with the integration of ESP32-based hardware. A custom Tasmota firmware image including TFLM support and Berry scripting language [29] was built to enable TinyML capabilities.

At this stage, the physical device must be connected to the hardware sensors (e.g., temperature, humidity, or vibration) that will provide data for the soft-sensor. To train the ML models, it is essential to establish a data stream where the device sends sensor readings to Kafka-ML. For supervised training, the target variable must also be measured. This establishes two different device roles within the architecture: "trainer" devices and "inference-only" devices. A trainer device is temporarily equipped with both the standard sensors and a high-fidelity (often expensive or complex) physical sensor to capture the

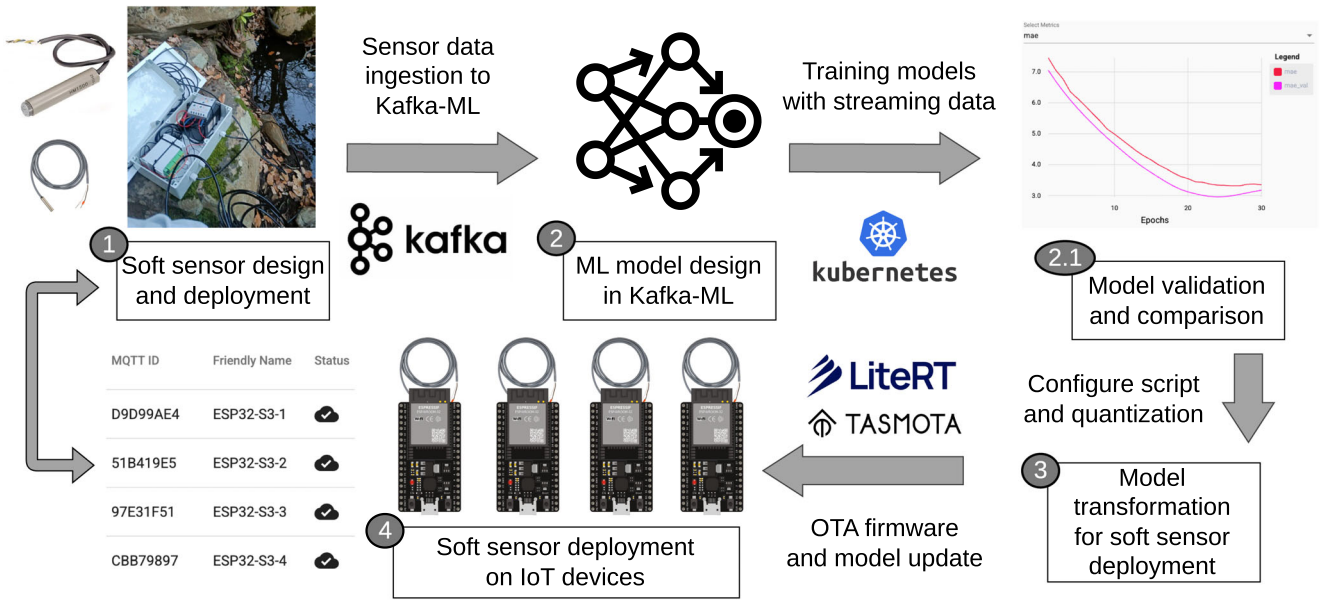


FIGURE 2 | Kafka-ML soft sensor deployment process pipeline.

ground-truth target data. For instance, in the water quality monitoring scenario, a trainer device would be temporarily equipped with both affordable multiparameter probes (measuring pH, temperature, etc.) and an expensive NO_3^- sensor. Once the model is trained on this combined data stream, the resulting soft sensor can be deployed to numerous inference-only devices. These target devices rely solely on standard, low-cost sensors to estimate the target variable locally, without requiring the specialized physical sensor. The 'IoT Manager' module orchestrates this registration by dynamically generating configuration scripts using Tasmota's *Backlog* command syntax. This transforms the IoT device into a soft-sensor-enabled unit, establishing a bidirectional data pipeline between the physical device and the Kafka-ML ecosystem.

4.2 | Training of ML Models in Kafka-ML

Once the device is registered and transmitting data, the ML model design phase starts. Kafka-ML collects the sensor streams and, when available, the target variables required for supervised learning.

Users can select these ingested streams to train a model within the Kafka-ML platform (refer to [7] for further details). This centralized training approach allows for high-performance computing resources to be used for model optimization before the model is transformed for the edge. Furthermore, the platform accommodates scenarios where live telemetry may be insufficient, such as cases involving poor data quality or missing target labels in the deployment environment. In such instances, Kafka-ML allows users to register datasets for the training process. This flexibility ensures that models can be initialized with robust parameters derived from high-quality offline data before being further refined or specialized using live IoT telemetry. Once the models have been trained, users can check the metrics on the same platform.

4.3 | Model Transformation for Soft Sensor Deployment

After training, the model must be prepared for the specific constraints of the ESP32. Deploying ML models on embedded devices often requires reducing their size to fit within limited RAM and Flash memory.

Kafka-ML and the 'IoT Manager' address this through an automated post-training 8-bit integer quantization and transformation into the TFLM format, executed by the *tfexecutor* module at the backend. For calibration, *tfexecutor* extracts 100 sequential samples from the recorded Kafka data stream. The converter tracks activation ranges across this dataset to compute scale factors and zero-points, mapping floating-point values to the INT8 [−128, 127] range. The pipeline applies a hybrid scheme: per-channel quantization for convolutional weights and per-tensor for dense layers and activations, with strictly enforced 8-bit integer inputs and outputs. This transformation significantly reduces the memory footprint and improves inference speed, ensuring that high-level cloud models are converted into deployment-ready binaries compatible with IoT devices. Once this phase is completed, the models are ready to be deployed on IoT devices.

4.4 | Soft Sensor Deployment and on-Device Inference

The final stage involves the OTA deployment and local execution. As in conventional TinyML pipelines, every trained model is converted into a quantized, TFLM-compatible artifact (Section 4.3) before it is deployed. Our contribution lies not in the inference mechanism itself but in how the artifacts are delivered and deployed at runtime. Instead of compiling the model into a monolithic firmware binary, our architecture ships the ML payload independently from the underlying host firmware. Deployment is

ALGORITHM 1 | OTA IoT Device Update Workflow

```
Input: Device dev, model, Berry script
Output: Predictions, actuator commands,
device status
function PREDICTANDACT (dev, model, data, script)
  prediction ← MakePrediction(model, data,
  script)
  if prediction means action then
     ActivateActuator(dev, script)
  end if
  return prediction
end function
# Main workflow
repeat
  operational ← GetDeviceStatus(dev)
until operational is true
repeat
  if receivedNewModel ≠ ∅ then
     DeployNewModelAndReboot(dev, model,
     script)
  ▷ triggers reboot; resumes below with new
   model/script
  end if
  data ← ReadSensorsData(dev, script)
  prediction ← PredictAndAct(dev, model, data,
  script)
untiloperational is false
```

coordinated via MQTT messages that convey HTTP URLs linked to the quantized model and the inference logic (Berry script). When these URLs are received, the IoT device downloads only the updated model binary and Berry script to its internal filesystem, then reboots to execute the Berry script. This hot-swappable approach eliminates the need for full firmware reflashing while simplifying deployment and model-update management in the field.

Algorithm 1 details the device operations for configuration, OTA updates, sensor reading, prediction, actuation, and status reporting, illustrating the interaction between the device and Kafka-ML. As part of this work, we have published a reference Berry script on the Kafka-ML GitHub repository,[§] serving as a customizable template for executing inference tasks on ESP32 devices.

To manage the transition between online and offline states, the system relies on the asynchronous network stack provided by the Tasmota firmware. The local inference loop, defined in the Berry script, operates independently of the network status. If the connection to the MQTT broker is lost, the device automatically transitions to an offline mode, continuing to read sensors and execute predictions locally. Meanwhile, the underlying firmware continuously attempts to restore connectivity in the background. It is important to note that while a human operator must explicitly request the initiation of a new model deployment or update via the Kafka-ML interface, the network recovery and operational fallback processes are fully autonomous. Once the network is recovered, the device automatically synchronizes with Kafka-ML to resume telemetry streaming and fetch any newly requested OTA updates, ensuring uninterrupted operational autonomy.

Figure 3 provides an overview of how input data can be connected to the deployed soft sensor, either from local sensors or external MQTT topics. This approach eliminates intermediate layers, ensuring low-latency inference by directly feeding sensor data into the model for inference. Sensor inputs are defined in the Berry script, which Kafka-ML updates seamlessly during the OTA deployment. As a result, predictions can be executed in real time, enabling immediate actuator response if needed.

Furthermore, the integration is not limited to local sensors. Input data can also be consumed from MQTT topics, enabling distributed soft sensors scenarios where measurements from multiple nodes are fused to estimate a single variable. In this case, the subscribed data replaces or complements local sensor values and is seamlessly injected into the inference pipeline, supporting flexible data fusion scenarios.

It should be noted that the Berry script governs this data handling and actuation logic at the application level only, delegating the inference computation entirely to the TFLM engine. The scripting layer thus provides deployment and runtime flexibility around a standard TinyML inference core, rather than altering the inference mechanism itself.

To sum up, this integration delivers a hybrid cloud-edge ML pipeline that bridges model development in Kafka-ML with on-device execution. This architecture facilitates the efficient deployment of TinyML soft sensors, ensuring high performance even in resource-constrained IoT devices.

5 | Performance Evaluation and Comparative Analysis

To demonstrate the versatility and robustness of the proposed approach for orchestrating end-to-end soft sensors at embedded devices, we designed our evaluation around two representative soft-sensing use cases: smart home energy monitoring [30] and real-time water quality monitoring [31]. Both scenarios leverage the same data ingestion, model training, quantization, deployment, and on-device inference pipeline, but target different sensing and inference goals.

5.1 | Use Cases and Datasets

The selected use cases serve to highlight the adaptability of soft sensor deployment automation across diverse sensing contexts and operational requirements. By focusing on scenarios with distinct data characteristics and inference objectives, the evaluation enables a comprehensive analysis of the system's performance under varying edge deployment conditions.

5.1.1 | Soft Sensor for Electrical Consumption Monitoring

In the first scenario, we conducted a case study focused on a Non-Intrusive Load Monitoring (NILM) task. In this context,

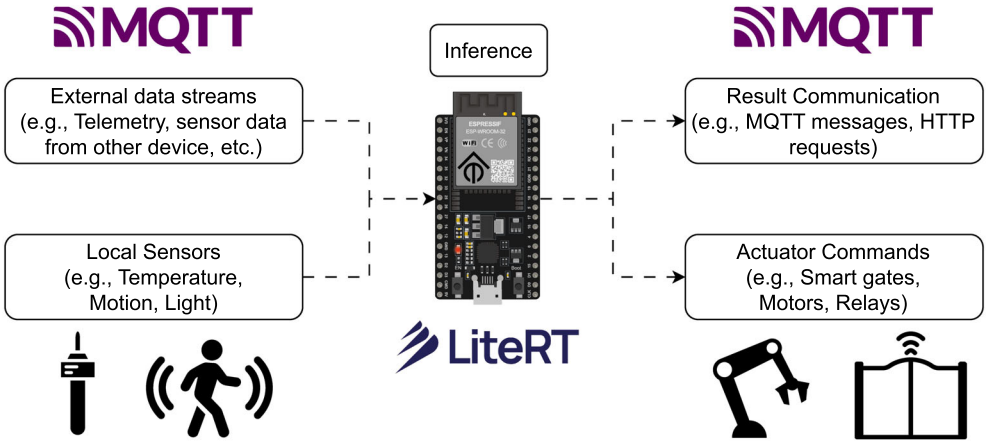


FIGURE 3 | Direct sensor input and MQTT-based input options for on-device inference.

the ESP32 acts as a virtual power meter, estimating the power consumption of individual appliances from aggregated household electricity measurements (e.g., fridge, oven and stove), an approach commonly used in real-world NILM applications to capture total load profiles.

The evaluation used the Individual Household Electric Power Consumption dataset [32], which contains over four years of minute-level power usage data from a single household. To replicate the deployment scenario, the data was streamed through Kafka-ML as if it were being captured in real time by the amperometric sensor. This configuration enabled end-to-end testing of the framework's data ingestion, processing, and deployment capabilities.

A Temporal Convolutional Network (TCN) [33] based model was trained using sequences of 120 time steps, each with two input features, global active power and global intensity. The model was designed to predict the disaggregated power usage of three target appliances. Its architecture consists of two residual TCN blocks with causal convolutions and dilation rates of 1 and 2, allowing the network to capture long-range temporal dependencies effectively. These are followed by a lightweight Conv1D output layer that predicts the power consumption of the three appliances, making the model suitable for embedded inference. This approach builds upon the foundational deep-learning baselines established for NILM tasks [34], adapting temporal architectures for efficient edge execution.

5.1.2 | Soft Sensor for Water Quality Monitoring

Ensuring the continuous assessment of water quality is a critical challenge, especially given the scarcity and cost of specialized sensors for certain contaminants, such as nitrates (NO_3^-). Traditional approaches often rely on costly, specialized sensors or persistent connectivity to cloud platforms, which may not always be feasible. Data-driven soft sensors, enabled by the integration of IoT devices and scalable machine learning frameworks, offer a compelling alternative for real-time monitoring of difficult-to-measure parameters. In this context, an embedded soft sensor system was implemented, prioritizing autonomous operation. By leveraging low-cost sensor arrays, the solution is capable of processing continuous water quality measurements and issuing real-time alerts even in circumstances where communication may be unreliable or latency is critical.

Open datasets such as the river water quality monitoring data from UK rivers spanning 1990 to 2018 were used [35]. This dataset includes extensive measurements of water quality parameters, including nitrate and turbidity, collected from multiple monitoring stations over a long temporal period. By streaming these data through Kafka-ML, the system simulates live sensor telemetry, enabling real-time testing of the soft sensor framework under realistic and continuous data flow conditions.

A fully connected feed-forward neural network [31] was trained to estimate NO_3^- concentrations from input sensor data. The model comprises three hidden layers with 128, 64, and 32 neurons, respectively, each employing the ReLU activation function to ensure compatibility with quantized deployment. A final linear output layer performs the regression task. Although dropout regularization was initially tested, it was excluded from the final configuration to enhance inference efficiency on embedded hardware.

5.2 | Experimental Setup

The experimental setup is supported by a high-performance infrastructure that ensures a thorough evaluation [36]. The following is a description of the system where the assessment took place:

- **Hardware Configuration**. The deployment of Kafka-ML has been performed on a Kubernetes cluster composed of 7 state-of-the-art servers. Each server features two Intel(R) Xeon(R) Gold 6230R CPUs, two NVIDIA(R) Tesla(R) V100 GPUs, and 384 GB of RAM, providing high-performance computing resources for model training and inference.

For the edge use case, an Adafruit HUZZAH32—ESP32 Feather Board is used, providing Wi-Fi connectivity and sufficient resources to run Tasmota firmware with TinyML inference.

- **Software configuration** Each node within the Kubernetes cluster runs K3s v1.28.3 on Ubuntu 22.04.3 LTS. Tasmota

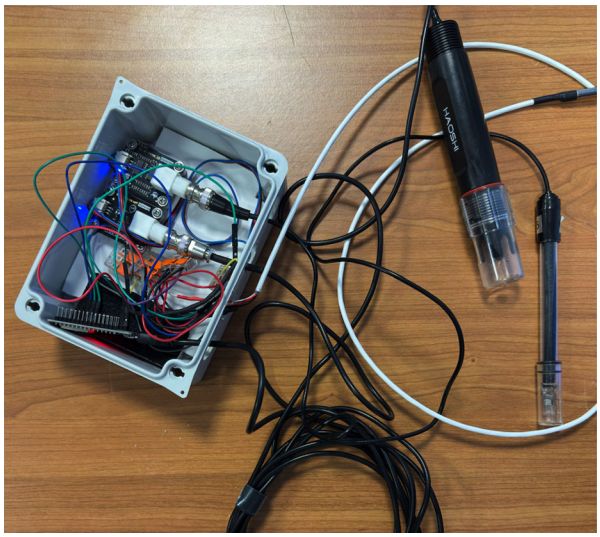


FIGURE 4 | Physical deployment prototype featuring an ESP32 microcontroller interfaced with water quality sensors, utilized to estimate real-time data acquisition overhead.

version 14.4.1.4 is compiled with custom flags to enable Berry scripting and TinyML capabilities on ESP32 devices.

This configuration extends the base Tasmota environment to incorporate TFLM support through the Berry scripting engine, allowing the deployment and execution of machine learning models directly on ESP32-based devices.

While the core evaluations rely on trace-driven data streams to ensure scientific reproducibility across extensive data distributions, a physical testbed was also constructed to validate the data acquisition phase in a real-world context (see Figure 4). Specifically for the water quality monitoring scenario, a suite of physical sensors, including a DFRobot EC Meter (DFR0300) for conductivity, a DFRobot pH Meter (SEN0169-V2) and an Adafruit DS18B20 temperature sensor, was interfaced with an ESP32 to infer NO_3^- concentrations, creating a functional prototype to evaluate the actual workflow and computational overhead of a soft sensor deployment in a controlled environment.

To support deployment on ESP32 microcontrollers running Tasmota, the model was converted to the TFLM format, quantized using 8-bit integer precision and deployed at device. These steps were fully orchestrated by Kafka-ML thanks to the new IoT Manager module.

The quantized models were deployed to ESP32 devices and performed on-device inference in real time using the streamed data. Performance metrics such as model size reduction, quantization accuracy loss and inference latency were monitored throughout the process.

5.3 | Training & Model Transformation Evaluation

For both use cases, training was performed using the Adam optimizer with a learning rate of 10^{-3}, with data shuffled at each epoch and a 70%–30% training–validation split. In the NILM task, the model was trained for 50 epochs with a batch size of 1024, minimizing the mean squared error (MSE) loss function. For the NO_3^- use case, training ran for 32 epochs with a batch size of 16, using the Huber loss function.

Quantization to 8-bit integer precision was validated by comparing the original floating-point model to the TFLM quantized version. As shown in Table 3, the performance difference between the regular and quantized models is almost negligible. For the NILM soft sensor, variations in MAE and MSE were below 0.01 while reducing model size from 41.69 KB to 22.15 KB. This confirms that the TCN architecture is highly robust to quantization and can be efficiently deployed on resource-constrained edge devices without compromising predictive accuracy.

Regarding the NO_3^- soft sensor use case, a measurable degradation is observed in terms of metrics, with the MAE rising from 0.53 to 0.82 mg/L (an approximate 55% relative increase). This phenomenon is well-documented in the TinyML field, where balancing model compression and performance is a recurring challenge. While the fine-tuning of such architectures to recover performance constitutes a substantial research effort in its own right, this work focuses on the validation of the complete end-to-end pipeline.

Consequently, the memory footprint was successfully reduced from 44.31 KB to 13.38 KB. Despite the accuracy degradation, the overall performance remains more than acceptable for this use case, providing a cost-effective estimation where physical sensing would be otherwise prohibitive.

The different effects of 8-bit quantization across the two use cases reflect the inherent resilience of the selected architectures. The NO_3^- soft sensor, based on a fully connected network, is more sensitive to quantization noise due to its narrow hidden layers and the small scale of its regression target (0–15 mg/L), where rounding errors have a higher proportional impact. In contrast, the NILM model utilizes a TCN, whose convolutional kernels and residual connections inherently mitigate precision loss through local averaging and information redundancy.

Consequently, while the TCN-based model shows negligible degradation, the MLP model exhibits a measurable but acceptable susceptibility to precision loss. For applications requiring higher fidelity in dense architectures, Quantization-Aware Training (QAT) [37] could be integrated into the Kafka-ML training stage as a future enhancement, mitigating precision loss by simulating quantization effects during the training forward pass.

Overall, these results demonstrate that the Kafka-ML framework can effectively train and deploy compact, accurate models for real-time energy disaggregation in smart home environments, as well as for NO_3^- concentration monitoring using soft sensors. This highlights the flexibility and adaptability of the framework, enabling efficient deployment across diverse edge computing scenarios where low-latency inference and resource-constrained environments are critical.

TABLE 3 | Comparison of model size, predictive accuracy, and response time between regular and quantized models for NILM and NO_3^- use cases.

Use case	Model type	Size (KB)	Training		Validation		Response time (ms)	
			MAE	MSE	MAE	MSE	Local	Remote
NILM	Regular (Float32)	41.69	2.821	26.628	2.836	27.001	—	710.9 ± 57.7
	Quantized (INT8)	22.15	2.821	26.625	2.837	27.006	154.9 ± 0.6	—
NO_3^-	Regular (Float32)	44.31	0.532	0.538	0.533	0.530	—	216.4 ± 41.8
	Quantized (INT8)	13.38	0.820	1.156	0.825	1.164	103.3 ± 0.5	—

5.4 | Prediction Latency and IoT Autonomy

Evaluating the latency of on-device versus remote inference is critical for determining the responsiveness and suitability of soft sensor deployment in practical edge scenarios. In our experiments, we successfully deployed the quantized models on ESP32 devices, enabling local inference without external dependencies.

To provide a meaningful comparison and characterize the system's variance, we measured the performance across $N = 100$ sequential samples in two scenarios:

- **Local Inference Latency:** The pure computational time required to execute the quantized model directly on the ESP32 using TFLM and Berry.
- **End-to-End Remote Response Time:** The total time elapsed from the moment the ESP32 transmits the sensor payload via MQTT to the moment it receives the predicted result back from the Kafka-ML endpoint. This metric includes network round-trip time, payload serialization, broker queuing, and cloud inference time. The evaluation was conducted over a standard WiFi local network using MQTT QoS 1, with a payload size of 960 bytes in NILM use case and 16 bytes for the NO_3^- use case.

The average prediction times for each use case, summarized in Table 3, demonstrate that on-device inference using the quantized model achieves significantly lower latency. Specifically, the NILM soft sensor responded 4.6× faster than the cloud-based alternative, while the NO_3^- soft sensor reduced latency by half. These findings highlight the advantages of deploying soft sensors directly on edge devices: reduced latency, increased autonomy, and improved responsiveness.

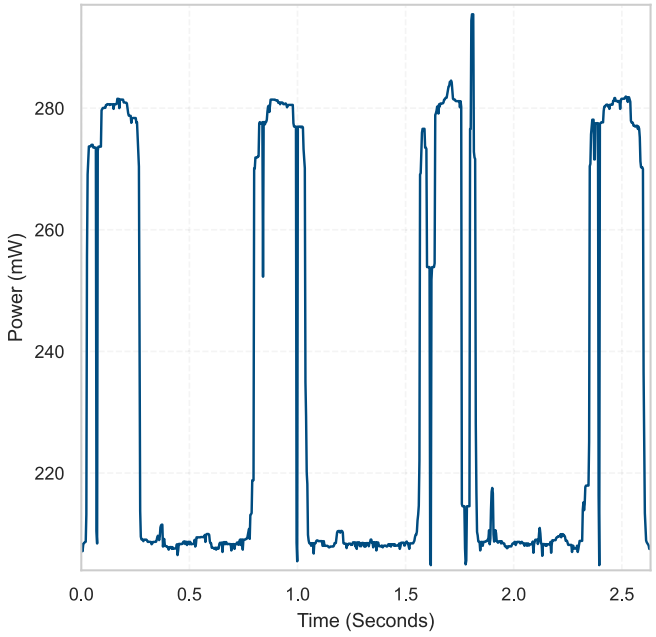
Furthermore, to characterize the complete physical execution cycle, the prototype shown in Figure 4 was utilized to evaluate the end-to-end sensing pipeline. The initial data acquisition phase, which encompasses reading raw sensor signals via the ESP32 GPIOs, applying basic preprocessing, and formatting the input tensors, introduced an overhead of 102.1 ± 1.2 ms. When this acquisition phase is executed sequentially with the core local inference, the system achieves a total end-to-end cycle time of approximately 205.4 ± 1.6 ms. This confirms that the physical integration of sensors seamlessly feeds the local model with minimal overhead (just a few milliseconds), validating the framework's real-world operational readiness.

5.5 | On-Device Energy Demand Analysis

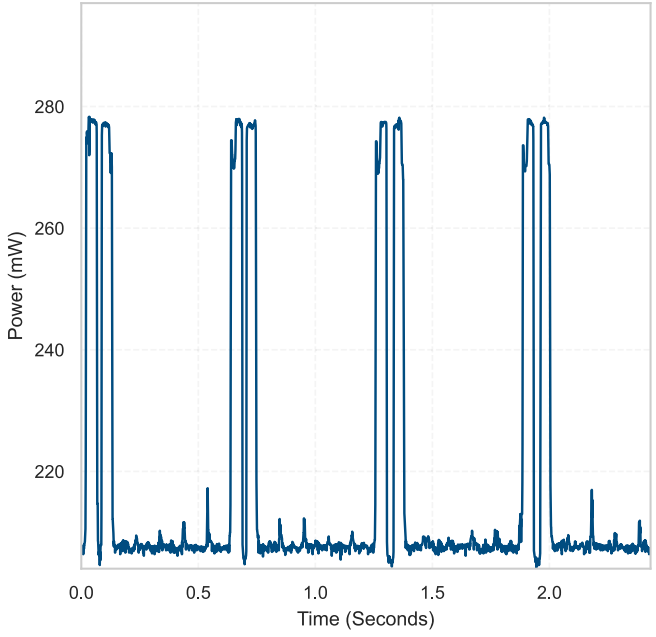
Beyond latency, the energy footprint of local inference is a critical factor for battery-powered or energy-sensitive edge deployments. The power profiles were captured using a Keysight N6705B DC Power Analyzer. Measurements were taken directly at the main power supply input of the ESP32-WROOM-32 module with a constant supply voltage of 3.3 V. The power profiles reflect the aggregate consumption of the entire microcontroller board. This includes the baseline power required to maintain active Wi-Fi connectivity and the MQTT network stack (approximately 208 mW), as well as the active on-board LEDs. To strictly isolate the computational energy footprint of the TinyML soft sensors, no external physical sensors were attached during this specific profiling phase. The raw telemetry was recorded during a continuous physical profiling session. The base sampling rates were set to approximately 341.45 Hz (interval of 2.93 ms) for the NILM use case and 1.68 kHz (interval of 0.59 ms) for the NO_3^- use case. To mitigate high-frequency measurement noise and accurately isolate the steady-state inference plateaus, a rolling window filter was applied to the raw signals prior to integration and visualization. Due to the highly deterministic hardware execution of the models, the filtered power signatures exhibited negligible variance across repetitions.

As illustrated in the power measurement profiles (Figure 5a,b), the energy footprint can be distinctly separated into network and computational phases. Both use cases exhibit a baseline consumption of approximately 208 mW during idle periods. This footprint is driven primarily by the active Wi-Fi modem, the MQTT stack required to maintain continuous connectivity, and the background execution of the Tasmota operating system, characterizing the ESP32 as a low-power, resource-constrained edge device. The minor fluctuations observed during this idle phase are standard network artifacts, primarily Wi-Fi beacons and MQTT keep-alive pings.

When an inference task is triggered, clear power spikes are observed, characterized by a steep ramp-up, a sustained plateau, and a ramp-down. The transition edges reflect the runtime overhead of the Berry scripting engine as it allocates the input tensors and subsequently parses the inference results. The central plateau represents the execution of the TFLM engine itself, driving the microcontroller's CPU cores to maximum utilization to perform the underlying matrix multiplications and convolutions. To rigorously quantify this energy demand, we calculated the total physical energy per inference E (in mJ) by performing a discrete integration of the power profile over the full duration of the



(a) NILM soft sensor use case.



(b) NO_3^- soft sensor use case.

FIGURE 5 | On-device inference power profiles by use case.

hardware event (from the initial power ramp-up to the return to the idle baseline):

- **NILM Use Case (TCN):** The inference cycles for the TCN-based model show a sustained power demand peaking at approximately 282 mW. The discrete integration of this physical event, which spans 257.7 ms, yields a total energy consumption of 69.71 mJ per inference. The plateau shape of these spikes reflects the computational intensity and longer execution time of the convolutional blocks and residual connections.
- **NO_3^- Use Case (MLP):** The fully connected network exhibits narrower power spikes, reaching similar peaks of approximately 278 mW. The integration of this shorter physical event, spanning 125.9 ms, reveals a significantly lower total energy consumption of 32.38 mJ per inference. This correlates with the lower local latency and the simpler matrix multiplication operations required compared to the NILM use case.

The findings indicate that while the peak power draw is similar across both model architectures, the total energy consumed per inference is significantly higher for the complex sequence modeling required in the NILM use case. This confirms that while the TCN architecture is highly robust to quantization, it imposes a measurable energy overhead that must be carefully considered when deploying battery-powered edge nodes.

5.6 | Orchestration Overhead and Fleet Scalability Analysis

To evaluate the operational efficiency and scalability of the proposed framework against traditional deployment methodologies, a fleet-wide deployment benchmark was conducted. Traditional TinyML workflows depend on compiling the model directly into the device firmware, a process that typically requires physical access and scales linearly ($O(N)$) with the number of nodes. For a fleet of 4 devices, physical flashing requires approximately 128 s of pure execution time: a one-time compilation step of 7.2 s plus 30.2 s of per-device firmware writing for each of the 4 devices ($4 \times 30.2\,\text{s} + 7.2\,\text{s} \approx 128\,\text{s}$), excluding manual cable swapping and port configuration.

In contrast, the proposed architecture decouples the ML payload from the host firmware, executing updates concurrently ($O(1)$) over-the-air via MQTT. To empirically test this behavior, a concurrent OTA deployment of the 22.15 KB quantized NILM model was triggered simultaneously across a physical testbed of multiple ESP32 devices over a standard Wi-Fi network.

Figure 6 illustrates the end-to-end OTA completion time, measured from the initial cloud deployment trigger to the successful hardware reboot and resumption of local inference, for configurations with 1, 2, and 4 concurrent devices. The experimental results demonstrate outstanding scalability, as the average deployment latency remains remarkably flat at approximately 20 s, regardless of whether a single device or the entire fleet is updated simultaneously.

Even when characterizing the system's jitter through the worst-case mean bounds (representing the slowest responding node in the concurrent batch), the latency overhead only shifts marginally to 20.5 s under full deployment load. This stable execution profile validates that the architecture does not introduce structural bottlenecks during fleet updates. Crucially, this performance demonstrates that leveraging a highly scalable production-ready technology stack, specifically Apache Kafka for high-throughput stream orchestration, MQTT for lightweight edge messaging, and Kubernetes for containerized cloud scaling, is precisely what makes this automated end-to-end orchestration feasible at scale, eliminating the constraints of traditional manual intervention.

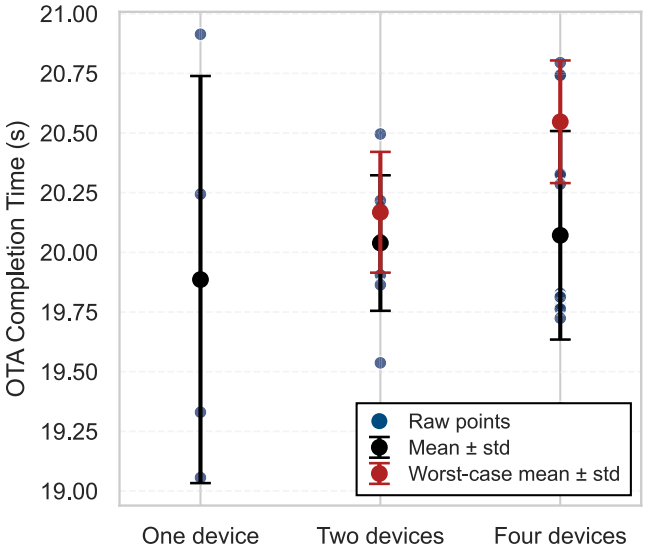


FIGURE 6 | End-to-end OTA deployment latency across different concurrent IoT fleet configurations (mean ± standard deviation and worst-case execution bounds).

6 | Conclusion and Future Work

This work presents a hybrid cloud-edge architecture for TinyML orchestration over real-time data streams, unifying ingestion, training, transformation, deployment, and on-device inference. The proposed framework is demonstrated through the deployment of autonomous soft sensors, addressing key requirements regarding operational efficiency and edge autonomy. This approach reduces latency, minimizes network dependence, and preserves privacy through local decisions at the edge while maintaining operational coherence from cloud to device. The pipeline employs Kafka-ML as the control plane and leverages OTA-enabled updates to deliver TinyML models and execute them on constrained hardware, removing the need for continuous connectivity and enabling responsive on-device decision-making.

The evaluation in NILM and water quality monitoring scenarios showcases the ability to deploy TinyML models with slight or minimal accuracy degradation and efficient inference performance. The modular architecture and the open-source nature of this approach ensure scalability and adaptability across diverse deployments. The successful orchestration of the ML lifecycle in a physical embedded environment using trace-driven telemetry confirms the feasibility and effectiveness of our approach for the democratization of edge intelligence.

Several promising directions can extend the capabilities of this framework:

- **Combining Federated Learning with TinyML Deployment**: Although Kafka-ML already supports federated learning [38], this new integration with embedded devices opens the door to collaborative soft-sensing workflows. Devices could participate in local training rounds receiving fine-tuned model updates over-the-air. This would enable personalized inference at the edge while preserving privacy and reducing communication costs. Some research has been done in this field with promising results [39].
- **Adaptive Deployment Based on Hardware Profiles**: Introducing mechanisms to automatically assess device-specific constraints (e.g., available RAM, flash storage, supported ops) would enable smarter model transformation pipelines that adapt quantization strategy, memory layout, or model architecture accordingly, ensuring stability and optimal performance across heterogeneous IoT fleets.
- **Expanding Firmware Compatibility**: The current implementation is limited to ESP32 devices running Tasmota firmware. Exploring support for other firmware platforms and extending compatibility to additional microcontrollers like STM32 or AVR would broaden the applicability of the system across a wider range of smart devices.
- **Large-Scale Empirical Fleet Validation**: While the Kafka-ML backend is built upon inherently scalable technologies (Kubernetes and Apache Kafka), our current edge evaluation focuses on validating the end-to-end pipeline of small physical IoT fleets. To conduct massive load testing, we are working on creating digital twins of embedded devices. This approach will allow us to seamlessly simulate firmware flashing and mimic the exact operational behavior of thousands of physical nodes. This will enable robust, large-scale empirical validation of OTA orchestration overhead and network congestion without the logistical constraints of massive physical hardware testbeds.

Author Contributions

Antonio Jesús Chaves: conceptualization, methodology, software, investigation, writing – original draft. **Cristian Martín:** conceptualization, validation, investigation, resources, writing – review and editing, supervision, funding acquisition. **Adnan Shahid:** validation, resources, writing – review and editing, supervision. **Manuel Díaz:** conceptualization, validation, resources, writing – review and editing, supervision, funding acquisition.

Acknowledgments

This work is funded by the Spanish projects: Grant PID2025-172820OB-I00 ('OPENTWINS-FED: Open Digital Twin Platform for Cross-Domain Simulation and Analytics of Federated and Collaborative Digital Twins Ecosystems') funded by MICIU/AEI/10.13039/501100011033/ and by 'FEDER'. Grant PLSQ_00230 ('iSAT: Sistema de Alerta Temprana Inteligente') funded by the Autonomous Government of Andalusia (Spain). Grant CPP2022-009695 ('TRIATHLON: TRIhAlomeTHanes controL thrOugh AI-based techNologies') funded by MICIU/AEI/10.13039/501100011033 and by 'European Union NextGenerationEU/PRTR'. This project has received funding from the European Union's Horizon Europe research and innovation programme under the Marie Skłodowska-Curie grant agreement EVOLVE No 101086218. Antonio Jesús Chaves is supported by the Spanish Ministry of Science, Innovation and Universities for the "Formación de Profesorado Universitario" (FPU) program under grant FPU23/00354. Funding for open access charge: Universidad de Málaga/CBUA.

Funding

This work was supported by Ministerio de Ciencia e Innovación, PID2025-172820OB-I00, CPP2022-009695, FPU23/00354, Junta de Andalucía, PLSQ_00230, and HORIZON EUROPE Marie Sklodowska-Curie Actions, 101086218.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available at [32, 35] and at Kafka-ML's Repository. Instructions to deploy Kafka-ML and all its code can be found as open-source at Kafka-ML's GitHub Repository (https://github.com/ertis-research/kafka-ml). These data were derived from the following resources available in the public domain: - Individual Household Electric Power Consumption, https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption, https://admin.opendatani.gov.uk/dataset/river-water-quality-monitoring-1990-to-20181.

Endnotes

[‡]https://github.com/ertis-research/kafka-ml.

[§]https://github.com/ertis-research/kafka-ml.

References

1. M. J. C. S. Reis and C. Serôdio, "Edge AI for Real-Time Anomaly Detection in Smart Homes," *Future Internet* 17, no. 4 (2025): 179.

2. T. Paepae, P. N. Bokoro, and K. Kyamakya, "From Fully Physical to Virtual Sensing for Water Quality Assessment: A Comprehensive Review of the Relevant State-of-the-Art," *Sensors* 21, no. 21 (2021): 6971.

3. M. G. S. Murshed, C. Murphy, D. Hou, et al., "Machine Learning at the Network Edge: A Survey," *ACM Computing Surveys (CSUR)* 54, no. 8 (2021): 1.

4. F. C. Andriulo, M. Fiore, M. Mongiello, E. Traversa, and V. Zizzo, "Edge Computing and Cloud Computing for Internet of Things: A Review," *Informatics* 11, no. 4 (2024): 71.

5. Google AI, "LiteRT for Microcontrollers," 2025, accessed April 28, 2025, https://ai.google.dev/edge/litert/microcontrollers.

6. R. David, J. Duke, A. Jain, et al., "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems," *Proceedings of Machine Learning and Systems* 3 (2021): 800.

7. C. Martín, P. Langendoerfer, P. Soltani Zarrin, M. Díaz, and B. Rubio, "Connecting the Data Stream With ML/AI Frameworks," *Future Generation Computer Systems* 126 (2022): 15.

8. A. J. Chaves, C. Martín, L. Llopis Torres, et al., "A Soft Sensor Open-Source Methodology for Inexpensive Monitoring of Water Quality: A Case Study of NO3− Concentrations," *Journal of Computational Science* 85 (2025): 102522.

9. N. Schizas, A. Karras, C. Karras, and S. Sioutas, "TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review," *Future Internet* 14, no. 12 (2022): 363.

10. V. J. Reddi, C. Cheng, D. Kanter, et al., "MLPerf Inference Benchmark," *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), Valencia, Spain* (2020), 446–459.

11. L. Han, Z. Xiao, and Z. Li, "DTMM: Deploying TinyML Models on Extremely Weak IoT Devices With Pruning," *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, Vancouver, BC* (2024), 1999–2008.

12. S. Jaiswal, R. K. K. Goli, A. Kumar, V. Seshadri, and R. Sharma, "MinUn: Accurate ML Inference on Microcontrollers," *Proceedings of the 24th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems, New York, NY* (2023), 26–39.

13. M. Carnelos, F. Pasti, and N. Bellotto, "MicroFlow: An Efficient Rust-Based Inference Engine for TinyML," *Internet of Things* 30 (2025): 101498.

14. B. Sudharsan, J. G. Breslin, M. Tahir, et al., "Over the Air Deployment of TinyML Models and Execution on IoT Devices," *IEEE Internet Computing* 26, no. 3 (2022): 69.

15. jomjol, "AI-on-the-Edge-Device Official GitHub Repository," 2020, accessed April 4, 2025, https://github.com/jomjol/AI-on-the-edge-device.

16. V. J. Reddi, A. Elium, S. Hymel, et al., "Edge Impulse: An MLOps Platform for Tiny Machine Learning," *Proceedings of Machine Learning and Systems* 5 (2023): 254.

17. Nordic Edge AI Lab, "Build Edge AI Models for Any Nordic Device," accessed July 7, 2026, https://ai.lab.nordicsemi.com/.

18. B. Saha, R. Samanta, S. Ghosh, and R. B. Roy, "BandX: An Intelligent IoT-Band for Human Activity Recognition Based on TinyML," *Proceedings of the 24th International Conference on Distributed Computing and Networking, New York, NY* (2023), 284–285.

19. S. O. Ooko, D. Mukanyiligira, J. P. Munyampundu, and J. Nsenga, "Edge AI-Based Respiratory Disease Recognition From Exhaled Breath Signatures," *2021 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT), Amman, Jordan* (2021), 89–94.

20. V. Tsoukas, A. Gkogkidis, E. Boumpa, S. Papafotikas, and A. Kakarountas, "A Gas Leakage Detection Device Based on the Technology of TinyML," *Technologies* 11, no. 2 (2023): 45.

21. O. Atanane, A. Mourhir, N. Benamar, and M. Zennaro, "Smart Buildings: Water Leakage Detection Using TinyML," *Sensors* 23, no. 22 (2023): 9210.

22. D. I. Németh and K. Tornai, "Fast Electrical Load Classification Using a Dimmer-Based Smart Plug," *Smart Cities, Green Technologies, and Intelligent Transport Systems, Communications in Computer and Information Science 1843* (2023): 68–89.

23. A. S. Pattanayak, B. S. Pattnaik, S. K. Udgata, and A. K. Panda, "Development of Chemical Oxygen on Demand (COD) Soft Sensor Using Edge Intelligence," *IEEE Sensors Journal* 20, no. 24 (2020): 14892.

24. B. S. Pattnaik, A. S. Pattanayak, S. K. Udgata, and A. K. Panda, "Machine Learning Based Soft Sensor Model for BOD Estimation Using Intelligence at Edge," *Complex & Intelligent Systems* 7, no. 2 (2021): 961.

25. E. A. M. Pereira, J. F. d. S. Santos, and E. d. A. Barboza, "An Energy Efficient TinyML Model for a Water Potability Classification Problem," *Sustainable Computing Informatics & Systems* 43 (2024): 101010.

26. A. Cerioli, L. Petrosino, D. Sasso, et al., "Efficient Detection of Microplastics on Edge Devices With Tailored Compiler for TinyML Applications," *IEEE Access* 13 (2025): 90970–90982.

27. A. Carnero, C. Martín, G. Jeon, and M. Díaz, "Online Learning and Continuous Model Upgrading With Data Streams Through the Kafka-ML Framework," *Future Generation Computer Systems* 160 (2024): 251–263.

28. Tasmota, "Tasmota Official GitHub Repository," 2025, accessed April 11, 2025, https://github.com/arendst/Tasmota/.

29. "The Berry Script Language," accessed August 3, 2025, https://berry-lang.github.io/.

30. R. Gopinath and M. Kumar, "A Case Study of Non-Intrusive Load Monitoring Edge Device in Commercial Building," *Energy and Buildings* 294 (2023): 113226.

31. A. J. Chaves, C. Martín, L. Llopis Torres, E. Soler, and M. Díaz, "A Methodology for the Development of Soft Sensors With Kafka-ML," In *Data Analytics for Internet of Things Infrastructure*, ed. R. Sharma, G. Jeon, and Y. Zhang (Springer, 2023), 307–324.

32. G. Hebrail and A. Berard, "Individual Household Electric Power Consumption," *UCI Machine Learning Repository* (2006), https://doi.org/10.24432/C58K54.

33. C. Lea, R. Vidal, A. Reiter, and G. D. Hager, "Temporal Convolutional Networks: A Unified Approach to Action Segmentation," *Computer Vision - ECCV 2016 Workshops, Lecture Notes in Computer Science* 9915 (2016): 47–54.

34. J. Kelly and W. Knottenbelt, "Neural NILM: Deep Neural Networks Applied to Energy Disaggregation," *Proceedings of the 2nd ACM International Conference on Embedded Systems for Energy-Efficient Built Environments, New York, NY* (2015), 55–64.

35. OpenDataNI, "River Water Quality Monitoring 1990 to 2018," http://data.europa.eu/88u/dataset/river-water-quality-monitoring-1990-to-2018.

36. ITIS Software, "ERTIS Research Fog Computing Cluster," 2024, accessed September 23, 2024, https://itis.uma.es/en/5437-2/.

37. B. Jacob, S. Kligys, B. Chen, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proceedings of the IEEE conference on computer vision and pattern recognition, Salt Lake City, UT* (2018), 2704–2713.

38. A. J. Chaves, C. Martín, and M. Díaz, "Towards Flexible Data Stream Collaboration: Federated Learning in Kafka-ML," *Internet of Things* 25 (2024): 101036.

39. Y. Li, R. Yao, D. Qin, and Y. Wang, "Lightweight Federated Learning for On-Device Non-Intrusive Load Monitoring," *IEEE Transactions on Smart Grid* 16, no. 2 (2025): 1950–1961.